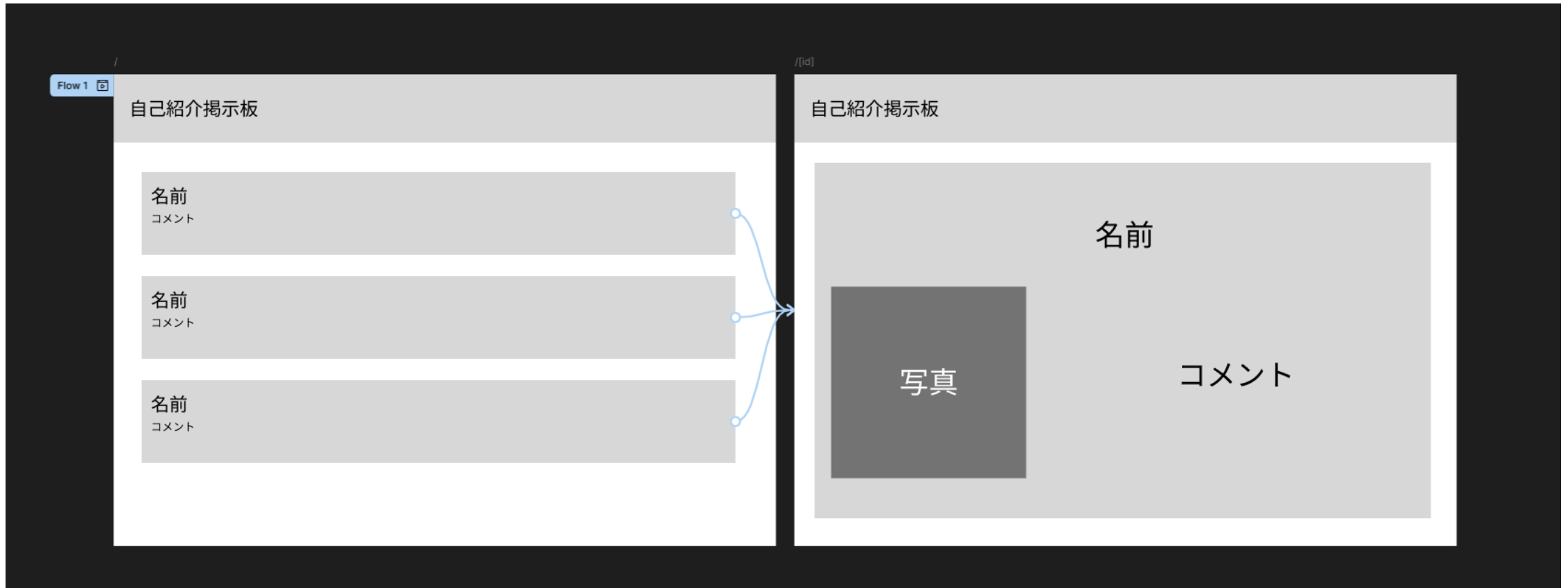


共通レイアウトとルーティング（画面遷移）

[ゴール]

全画面共通のレイアウト設定と、
URLに応じた画面の切り替え（ルーティング）を理解する。

一旦、全体像の確認



実装1：全画面共通のヘッダーを作る

component/Header.tsx

```
import styles from "../Header.module.css";

interface HeaderProps {
  title: string;
}

const Header = (Props: HeaderProps) => {
  return (
    <div className={styles.Header}>
      <div className={styles.title}>
        <p className="">{Props.title}</p>
      </div>
    </div>
  );
};

export default Header;
```

component/Header.module.css

```
.Header {  
  margin: 10px 0px;  
  border-top: 4px solid #000;  
  border-bottom: 4px solid #000;  
}  
  
.title {  
  padding: 15px;  
  padding-left: 10px;  
}
```

実装2：layout.tsxを使った共通化

layout.tsx（一部抜粋）

```
<body>  
  <Header title="自己紹介掲示板" />  
  {children}  
</body>
```

実装3：クリックで画面遷移させる準備

component/UserCard.tsx

```
import styles from './UserCard.module.css';
import Link from 'next/link';

interface UserCardProps {
  id: string;
  name: string;
  comment: string;
}

const UserCard = ({ id, name, comment }: UserCardProps) => {
  return (
    <Link href={`/${id}`}>
      <div className={styles.card}>
        <h1 className={styles.name}>{name}</h1>
        <p>{comment}</p>
      </div>
    </Link>
  );
};
export default UserCard;
```

Next.jsのルーティング（ディレクトリ構造）

URLとフォルダ構成は連動する

```
app/  
├── page.tsx      <-- http://localhost:3000/（一覧画面）  
└── [id]/  
    └── page.tsx  <-- http://localhost:3000/1（詳細画面）
```

[id] とは？

- 動的ルーティング（Dynamic Routing）と呼ばれる仕組み
- **1** や **2** など、アクセスされたURLに応じて中身が変わる

自己紹介掲示板

田中 太郎

数学が大好きです

田中 二郎

ラーメンが大好きです

田中 三郎

三河屋で働いています

自己紹介掲示板

田中 太郎

花子さんと太郎さんは、グラフ表示ソフトを用いて、
とおいた2次関数 $y = x^2 + px + q$ と $y = x^2 + qx + p$
考えている。



数学が好きです

自己紹介掲示板

田中 二郎



ラーメンが好きです

実装4：詳細カードコンポーネントの作成

component/DetailCard.tsx

```
import Image from "next/image";
import styles from "@component/DetailCard.module.css";

interface UserDetailProps {
  image: string; name: string; comment: string;
}

const DetailCard = ({ image, name, comment }: UserDetailProps) => {
  return (
    <div className={styles.container}>
      <div className={styles.nameSpace}>
        <span className={styles.name}>{name}</span>
      </div>
      <div className={styles.innerContainer}>
        <div className={styles.imageSpace}>
          <Image src={image} alt={name} width={300} height={150} className={styles.image} />
        </div>
        <div className={styles.commentSpace}>
          <span className={styles.commet}>{comment}</span>
        </div>
      </div>
    </div>
  );
};
export default DetailCard;
```

component/DetailCard.module.css

```
.container {
  border: 4px solid #000;
  border-radius: 24px;
  padding: 4px;
}
.nameSpace {
  display: flex;
  justify-content: center;
  border-bottom: 4px solid #000;
  padding: 4px;
  margin-bottom: 4px;
}
.name {
  text-align: center;
  font-size: 24px;
}
.innerContainer {
  display: flex;
}
.imageSpace {
  margin-right: 4px;
  border: 4px solid #000;
  border-radius: 24px;
  overflow: hidden;
}
.image {
  width: 100%;
  height: 100%;
}
.commentSpace {
  border: 4px solid #000;
  border-radius: 24px;
  flex: 1;
  padding: 6px;
}
.comment {
  font-size: medium;
}
```

実装5：詳細画面（page.tsx）の完成

app/[id]/page.tsx

```
import DetailCard from "@component/DetailCard";
import styles from "@app/[id]/Page.module.css";
import { users } from "@data/user"; // モックデータ

interface DetailPageProps {
  params: { id: string };
}

const DetailPage = async ({ params }: DetailPageProps) => {
  const { id } = await params;
  const user = users.find((user) => user.id === id); // 本来はここでDBから取得する

  if(!user) return <div>ユーザーが見つかりません</div>

  return (
    <div className={styles.main}>
      <DetailCard image={user.imageUrl || ""} name={user.name || ""} comment={user.comment || ""} />
    </div>
  );
}

export default DetailPage;
```

app/[id]/Page.module.css

```
.main {  
  padding: 4px;  
}
```

モックデータの型と中身

data/user.tsx

```
export interface UserData {  
  id: string;  
  name: string;  
  comment: string;  
  imageUrl?: string;  
}  
  
export const users: UserData[] = [  
  { id: "1", name: "田中 太郎", comment: "数学が大好きです", imageUrl: "/taro.png"},  
  { id: "2", name: "田中 二郎", comment: "ラーメンが大好きです", imageUrl: "/jiro.jpg"},  
  { id: "3", name: "田中 三郎", comment: "三河屋で働いています", imageUrl: "/saburo.png"},  
];
```

まとめ

- 共通レイアウト（`layout.tsx`）
 - `{children}` を活用することで、ヘッダーなどを全画面に一括で適用できる。
- ルーティング（画面遷移）
 - Next.jsはフォルダ名がそのままURLになる。
 - `<Link>` コンポーネントを使うことで、画面遷移を実装できる。
- ダイナミックルーティング（`[id]`）
 - 動的に変わるURLに対応できる。
 - URLから情報（id）を受け取り、それをもとにデータを取得して表示するのが実務での基本。